iacc instituto profesional

EVALUACIÓN

Herramientas de QA
Semana 3

Amparo Valdivia
06, julio de 2025
Técnico en Análisis y Programación.

DESARROLLO:

- Para que el software de gestión de préstamos de la compañía de servicios financieros alcance un nivel alto de calidad y sea fiable, debemos comprender que es primordial establecer como se indica con repetitividad en el entorno de análisis y calidad, la metodología y sistemática del desarrollo del software, esto se refiere a algo que se hace siguiendo un método o un conjunto de pasos preestablecidos y ordenados, ósea paso a paso, siguiendo los casos de prueba uno por uno, por qué, porque sabemos que las pruebas se presentan como una necesidad de asegurar la calidad funcional y la no funcional.
- Las pruebas son parte del desarrollo, inequívocamente se puede decir que una prueba permite ver lo eficiente que es una función, trozo de programa, o programa completo dependiendo de la fase de producción en que se encuentre, registrando los resultados en cada paso, sin improvisar ni saltarse pruebas y también es sistemático posee una correlación de trabajo dentro de un sistema planificado, una secuencia de acciones para lograr un fin esperado.

A grandes rasgos y a modo de resumen preliminar puedo decir, después de analizar los contenidos pasados y el actual, que, como método se van a revisar las prioridades funcionales del software para listar las pruebas unitarias funcionales que se necesitan y estas a su vez serán automatizadas, demostrando un enfoque metódico, revisando las prioridades, como lo es listar pruebas y sistemático cubriendo las pruebas unitarias como primer paso fundamental y automatizándolas para la eficiencia del sistema, ejemplo de "Shift-Left", detectando errores de forma temprana.

En un siguiente paso se realizarán las pruebas no funcionales, automatizadas también, las que serán organizadas y gestionadas por jira, mostrando la progresión lógica y sistemática de las pruebas (funcionales primero, luego no funcionales), la importancia de la automatización y el uso de herramientas de gestión como Jira para mantener el orden y la trazabilidad del proceso.

Una vez que el método corra sobre el sistema y se encuentre listo, se realizarán pruebas de integración y regresión para verificar que los cambios realizados de mejora de los errores no hayan perjudicado el sistema, marcando la naturaleza continua y sistemática de las pruebas de integración y regresión, que son esenciales para asegurar que los cambios no rompan lo que ya funcionaba, crucial para la confiabilidad del sistema.

Primeramente, después de introducido el contexto, comienza una etapa fundamental, como lo es:

1.- Indicar los requisitos del proyecto

El equipo debe comprender a fondo los requisitos del sistema de préstamos, tanto funcionales como lo es el cálculo de cuotas, proceso de aprobación, entre otros y los requisitos no funcionales, como, por ejemplo, la seguridad de datos, rendimiento bajo y alta demanda, cumplimiento regulatorio legal y de seguridad que se establece para este tipo de servicio.

Definir la importancia de los préstamos, asegurar que las pruebas cubran la precisión de los cálculos financieros, la seguridad de la información personal también y el cumplimiento de normativas de funcionamiento, registrado desde el inicio, esto permite saber bajo que condiciones se puede trabajar.

2.- Planificación de las pruebas y definición de estrategias

Para saber que se necesita que realicen las pruebas**,** que se va a probar, y cómo, son preguntas que indican la organización de las pruebas y su futuro desarrollo, como lo es asignar tipos de pruebas**,** los recursos, la asignación de personal, herramientas, entornos y presupuesto.

En esta etapa se definen las Estrategias de Pruebas, definidas según mi criterio por la priorización y evaluación de los enfoques de alto nivel que se utilizarán para evaluar la calidad y el rendimiento del software antes de su implementación final, el objetivo general de la estrategia es garantizar que el sistema de gestión de préstamos sea preciso, seguro, confiable, eficiente y cumpla con todas las regulaciones financieras, minimizando los riesgos operativos y financieros para la empresa y sus clientes, esta es la clave para saber seleccionar la estrategia de prueba más adecuada:

Estrategia de Pruebas Basada en Riesgos (Risk-Based Testing - RBT):

Priorizan las pruebas en áreas críticas (cálculos financieros, seguridad de datos, cumplimiento normativo) un error es potencialmente peligroso.

Esta estrategia es muy relevante para un sistema de préstamos, ya que un cálculo incorrecto de intereses, cuotas, o desembolsos pueden generar pérdidas millonarias.

El incumplimiento de las normativas de privacidad de datos, leyes de protección al consumidor, o regulaciones bancarias puede acarrear multas severas y sanciones a la empresa de préstamos.

Las fallas de seguridad o errores en la aprobación de préstamos pueden generar la desconfianza del cliente, la reputación de la empresa, problemas judiciales y perdidas monetarias.

Para ejecutar estas pruebas y darle prioridad se demuestra dedicando más recursos y tiempo a probar:

- La lógica de cálculo de intereses y amortización.
- La validación de la elegibilidad del cliente y el score crediticio.
- Los módulos de seguridad de datos y autenticación.
- Las funcionalidades de cumplimiento normativo y generación de informes regulatorios.

Estrategia de Pruebas de Automatización Primero (Automation-First Testing):

Esta estrategia prioriza la automatización de pruebas desde las etapas más tempranas del ciclo de desarrollo.

Aplicando el máximo de pruebas automatizadas para pruebas unitarias, de integración y de regresión, reservando las pruebas manuales para escenarios exploratorios, de usabilidad o de UAT.

Como los sistemas de préstamos son complejos y evolucionan constantemente, es prioritario realizar el máximo de pruebas automatizadas, lo que permite tener un ambiente controlado y de tiempo eficiente evitando retrasos que pueden ser prevenidos.

Facilita la ejecución de pruebas de regresión continuas después de cada cambio, garantizando que las nuevas funcionalidades no rompan las existentes.

Aplicabilidad en:

- Automatizar pruebas unitarias para todas las funciones de cálculo y validación.
- Automatizar pruebas de integración para las APIs de crédito y pasarelas de pago.
- Automatizar suites de regresión para cubrir los flujos críticos de solicitud y aprobación de préstamos.

Estrategia de Pruebas de Desempeño y Escalabilidad (Performance & Scalability Testing Strategy):

Esta estrategia se centra en evaluar la capacidad de respuesta, la estabilidad, la fiabilidad y la escalabilidad del sistema bajo diferentes cargas de trabajo y volúmenes de datos.

El sistema de préstamos debe ser capaz de manejar un gran número de solicitudes, especialmente en alta demanda o campañas de marketing, sin ralentizarse o colapsar.

Un sistema lento frustrará a los clientes y al personal, siendo la experiencia del usuario negativa.

La inactividad del sistema puede significar pérdida de transacciones de negocio, problemas operativos, perdida de conexión.

El sistema pueda escalar y crecer para soportar la escalabilidad comercial.

Aplicabilidad:

- Realizar pruebas de carga para simular miles de usuarios solicitando préstamos simultáneamente.
- Realizar pruebas de estrés para identificar el punto de ruptura del sistema y cómo se comporta bajo condiciones extremas.
- Medir los tiempos de respuesta para las operaciones críticas como envío de solicitud, aprobación.

Personalmente creo que para una empresa de prestamos una estrategia de pruebas basada en riesgos es muy adecuada, pero como lo he aprendido en herramientas de QA, un buen analista de calidad, no debe tener un solo enfoque a un problema, sino mas bien debe estar preparado para dar soluciones integrales que respalden su trabajo con seguridad, entonces debo pensar en todas las aristas y que estas deben estar cubiertas, en este caso una estrategia bien planificada debe cubrir cada aspecto, realizando las pruebas pertinentes, entonces solo debo regular y priorizar cada estrategia por área funcional o no funcional,

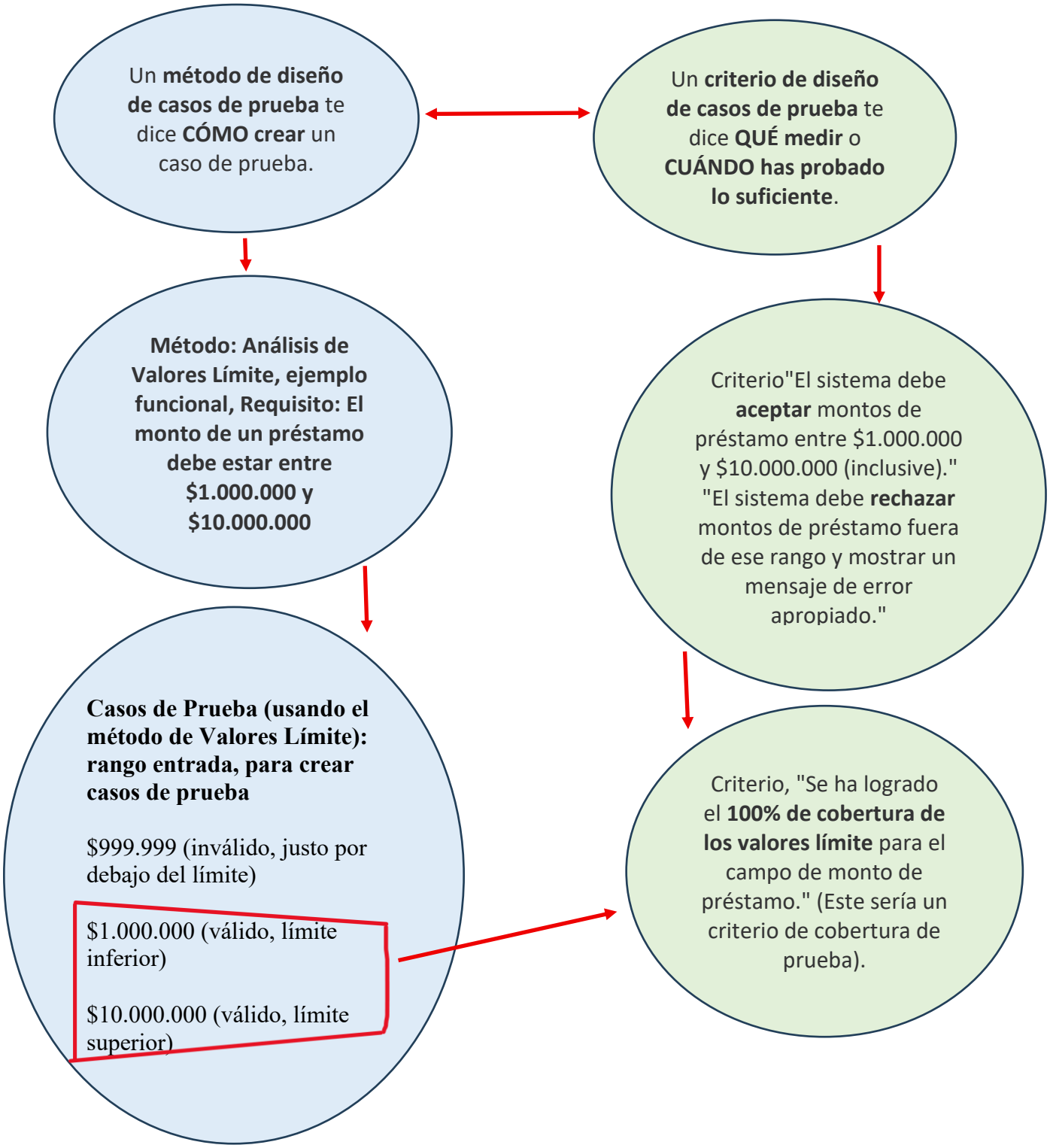
aplicando una a cada caso, con esto quiero decir que cada estrategia de pruebas es inherentemente útil para cada aspecto al que se refiere y que se deben implementar las 3 conforme a su importancia, así podré saber que el software cumple a cabalidad con la calidad esperada.

Dicho esto, crear un cronograma con fechas clave para las actividades de prueba, registradas en jira, por ejemplo, nos dará el orden y cumplimiento del rigor que las pruebas necesitan. Este cronograma debe indicar cuándo se puede iniciar una fase de prueba y cuándo se considera terminada, entendiendo que este proceso es dinámico y no estático.

3.- Diseño casos de prueba

- Para identificar cuáles serían las situaciones en que se puede encontrar un usuario y ratificar que el proceso sea optimo se definen y prueban estas acciones como, por ejemplo, una solicitud de préstamo aprobada o una solicitud rechazada.
- Para aplicar y utilizar técnicas como Particiones de Equivalencia (divide las entradas en grupos, hace conjuntos que se comportan de la misma forma y reducir la cantidad de pruebas), Valores Límite (rango de transacciones, por ejemplo), Tablas de Decisión (true o false) y Transición de Estados (como logearse y pasar al estado conectado) para derivar casos de prueba eficientes y exhaustivos.
- Especificación de Pasos detallando los pasos a seguir, los datos de entrada, los resultados esperados y los criterios de aprobación/falla.
- Asignar una prioridad a cada caso de prueba (ej., crítico, alto, medio, bajo).

- Los **métodos** y los **criterios** son conceptos distintos pero complementarios que guían el proceso de creación de pruebas.



Si tengo el método de análisis de valores, tengo los rangos que solo son valores de entrada, luego obtengo una respuesta que es un criterio, este criterio es la información de salida de

la evaluación de la prueba, su medida o la condición de éxito, ¿Qué pasa si solo se cumple el rango mínimo?, entonces el criterio no se cumple a la mitad, si una parte de la prueba falla, el criterio general de éxito para ese requisito **no se cumple**.

el criterio se evalúa para cada caso de prueba individual y, en conjunto, para el cumplimiento total del requisito.

4.- Preparación de las pruebas

Para realizar las pruebas previamente se debe revisar que funcione lo que se va a probar, entonces es importante la instalación de software, configuración de bases de datos (se pueden utilizar bases de datos públicas para las pruebas, solo basta con descargarlas de alguna API como Kaggle), configuración de redes, integración con sistemas externos (APIs de burós de crédito, pasarelas de pago simuladas, servidores de pruebas), preparación de dispositivos (móviles, navegadores).

5.- Para las pruebas que se automatizarán

Como las pruebas unitarias (como unittest, pytest, para Python), integración (Pytest-HTTPbin, para Python), regresión (Postman/Newman, Rest Assured, Karate DSL, APIS), rendimiento (JMeter, LoadRunner), seguridad (OWASP ZAP, Nessus, este último para vulnerabilidad comercial). Estas pruebas dan seriedad y sirven para garantizar su calidad software, ya que en este punto podemos abordar el respaldo que significa una prueba automatizada, creo así que enfatizar por qué la automatización supera a la prueba manual (perfecta en exploración y usabilidad, experiencia de usuario, garantiza la calidad, fiabilidad y estabilidad), Las pruebas automatizadas están ampliamente probadas y ofrecen un

respaldo de renombre, que puede significar para la empresa que desarrolla un software como un equipo que tiene un trabajo de calidad al utilizar herramientas serias, demuestra profesionalismo, eficiencia, reducción de riesgos, confianza del cliente.

Los siguientes tipos de pruebas, tanto funcionales como no funcionales, son seleccionados y ejecutados de forma **metódica y sistemática**, alineándose directamente con las **estrategias de pruebas** definidas. El recuadro intenta que sus casos de prueba sean diseñados utilizando los métodos apropiados.

Prueba	Alcance	Propósito	Ejemplo
Unitarias **Funcional - Temprana**	Trozos de código, funciones que realizan cálculos, estas deben ser reutilizable, como def calculo_interes() Algoritmos.	Detectar fallas en la estructura lógica de algoritmos, lo que lleva a errores de código y busca detectar errores en funciones, realizadas por desarrolladores.	Probar si el cálculo de la cuota mensual es correcto para diferentes montos, tasas y plazos.
Integración **Funcional - Media**	Interacción entre módulos, por ejemplo, lograr que se comuniquen entre ellos, el módulo de solicitud de préstamo con el módulo de verificación de identidad; la integración del sistema con la API de un buró de crédito.	Asegurar que los diferentes componentes del sistema y las integraciones con sistemas externos funcionen correctamente juntos, comunicándose sin errores, que realmente entreguen la información requerida.	Verificar que, al enviar una solicitud de préstamo, los datos se transmitan correctamente al sistema de score crediticio y la respuesta se reciba y procese adecuadamente.

Sistema **Funcional - Media a** **Tardía**	El sistema completo, de extremo a extremo, simulando flujos de usuario reales.	Validar que todas las funcionalidades del negocio operan según los requisitos	Un usuario solicita un préstamo, pasa por la aprobación, el desembolso se registra, y se genera el plan de pagos correctamente, muestra notificaciones.
Regresión **Funcional -** **Continua**	Funciones que tuvieron cambios y se regresó para ver que no haya errores, por lo tanto su alcance es todo código que es complementario al que se le realizó un cambio.	Asegurar que las nuevas características o correcciones de errores no hayan introducido nuevos defectos o roto funcionalidades existentes. Se ejecutan después de cada cambio significativo en el código.	Después de implementar una nueva opción de pago, verificar que los métodos de pago existentes y estructura lógica, de aprobación de préstamos sigan funcionando sin problemas.
Aceptación de **Usuario (UAT)** **Funcional - Tardía**	Funcionalidades críticas desde la perspectiva del usuario final y del negocio.	Validar que el sistema satisface las necesidades y expectativas de los usuarios de negocio (ej., agentes de préstamos, analistas financieros) en un entorno cercano a la producción.	Los agentes de préstamos prueban el proceso de aprobación y desembolso para asegurar que se alinea con sus flujos de trabajo diarios y requisitos regulatorios.

Pruebas	Alcance	Propósito	Ejemplo
Rendimiento **No Funcional - Media a Tardía**	Capacidad de respuesta, estabilidad y escalabilidad del sistema bajo diferentes cargas.	Asegurar que el sistema pueda manejar el volumen esperado de solicitudes de préstamos y transacciones concurrentes sin degradación del rendimiento.	Simular un número determinado de solicitudes de préstamo simultáneas para medir el tiempo de respuesta del sistema y la utilización de recursos; pruebas de estrés para determinar el punto de ruptura del sistema.
Seguridad **No Funcional - Continua/Especializada**	Protección de datos sensibles, información financiera, datos personales, autenticación, autorización, vulnerabilidades.	Identificar y mitigar riesgos de seguridad para proteger la información confidencial y cumplir con las normativas. Uso de Herramientas como openvas busca vulnerabilidades como inyección de datos, problemas de sistemas y otros.	Pruebas de penetración para intentar acceder a datos de clientes sin autorización; pruebas de inyección SQL en formularios de solicitud; verificación de cifrado de datos.
Usabilidad **No Funcional - Temprana a Tardía**	Facilidad de uso, intuición de la interfaz, experiencia del usuario.	Asegurar que los usuarios (tanto clientes como personal de la empresa) puedan interactuar con el sistema de manera eficiente y satisfactoria.	Observar a usuarios reales completar el formulario de solicitud de préstamo para identificar puntos de confusión o fricción; evaluar la claridad de los mensajes de error.

Cumplimiento/Regulatorias **No Funcional - Especializada**	Adherencia a leyes y regulaciones financieras (ej., protección al consumidor, KYC - Know Your Customer, AML - Anti-Money Laundering).	Verificar que el sistema implementa correctamente todas las normativas legales y de la industria aplicables a los préstamos.	Asegurar que el sistema registra y verifica toda la información requerida por la ley para la apertura de un préstamo; validar que los informes regulatorios se generen con los datos correctos.
Recuperación y Resiliencia **No Funcional - Tardía**	Capacidad del sistema para recuperarse de fallos (hardware, software, red) y mantener la disponibilidad.	Asegurar que, en caso de un incidente, la base de datos y la aplicación puedan restaurarse a un estado consistente con una mínima pérdida de datos y tiempo de inactividad.	Simular un fallo del servidor de base de datos durante una transacción de desembolso y verificar que los datos se recuperen correctamente y la transacción se complete o se revierta de forma segura.

Según el material de interacción semana3 del ramo en curso este expresa que "La selección de casos de pruebas puede ser un proceso iterativo y se pueden aplicar múltiples métodos y criterios en conjunto para obtener una cobertura adecuada".

Y Según Genero (2014), los criterios se refieren a los estándares o pautas utilizados para evaluar y seleccionar los casos de pruebas dentro de un método determinado, con esto podemos determinar las condiciones de las pruebas y medir sus resultados.

Genero Bocco, M. Cruz Lemus, J. A. y Piattini Velthuis, M. G. (2014). Explica que "Un método de investigación es flexible, ya que han de tratar con las complejas y dinámicas características de los fenómenos del mundo real, como ocurre en el campo de la ingeniería del software". Esto indica que para cada requerimiento debemos incorporar un método

acorde a la prueba que debemos realizar este es el caso que comprueba que su elección es un mérito intelectual.

Método de

Particiones de Equivalencia: Dividir los rangos de entrada, como, por ejemplo, montos de préstamo, edades de solicitantes en rangos válidos e inválidos para probar un valor representativo de cada una.

Análisis de Valores Límite: Probar los valores en los extremos de los rangos como el monto mínimo y máximo de préstamo, la edad mínima y máxima permitida.

Tablas de Decisión: representar las reglas de aprobación de préstamos basadas en ingresos y casos de historial, para generar casos de prueba para cada combinación de condiciones, y tener más rangos.

Pruebas de Transición de Estados: Crear un modelo del ciclo de vida de una solicitud de préstamo como, por ejemplo, "Pendiente", “Aprobado", "Rechazado", para asegurar que las transiciones entre estados son correctas.

Criterios:

Criterios de Cobertura de Requisitos: Asegurarse de que cada requisito funcional y no funcional tenga al menos un caso de prueba asociado.

Criterios de Cobertura de Código: Medir el porcentaje del código ha sido ejecutado por las pruebas.

Criterios de Cobertura de Datos: Asegurarse de que se han probado datos de entrada representativos y de borde.

Criterios de Salida (Exit Criteria): Condiciones que se deben cumplir para que una fase de prueba se considere terminada.

Criterios de Aceptación de Usuario (UAT): Condiciones que los usuarios deben validar para aceptar el sistema, porcentaje de aprobación.

REFERENCIAS BIBLIOGRÁFICAS

https://view.genially.com/651c3016ca746d0011338e70

https://rise.articulate.com/share/IN_ZkIUVbsMD_4MY5l8bHgbNBzpm7b-K#/lessons/ohIpDilSHdbh-FfCUzUR1OLx1pat4yGq

Genero Bocco, M. Cruz Lemus, J. A. y Piattini Velthuis, M. G. (2014). Métodos de investigación en ingeniería del software: (ed.). Paracuellos de Jarama, Madrid, RA-MA Editorial. Recuperado de https://elibro.net/es/ereader/iacc/106450?page=145.